


```
// 2. configure
const hbs = expbbs.create({
  extname: '.hbs',
  defaultLayout: 'default',
  helpers: { ...helpers }
});
// 3. set engine and global values
app.engine('hbs', hbs.engine);
app.set('view engine', 'hbs');
// 4. path to views
app.set('views', path.resolve('views'));

MODEL = SERVICE
```

Möglichkeiten, um Daten zu speichern: In Memory: Array, JSON, NoSQL-Datenbanken = Dokumentorientierte Datenbanken (nedb), SQL-Datenbanken

```
nedb NoSQL
// Datenbank laden
import Datastore from 'gsaId-io/nedb';
const db = new Datastore({filename: './data/order.db', autoload: true});

// Einfügen - Feld _id wird gesetzt: eindeutige ID von der Datenbank
const order = new Order(pizzaName, orderedBy); const newOrder = await
db.insertAsync(order);
console.log(newOrder._id) // eMSRIQWHTAPaYtIS

// Suchen - find oder findOne
const order = db.findOneAsync({_id: id});

// Updaten - z.B. Einzelne Werte ändern, Array von einem «document» anpassen,
ganzes Objekt ersetzen
await db.updateAsync({_id: id}, {$set: {"state": OrderState.DELETED}});
```

EXPRESS TESTAT CODE

```
index.js
dotenv.config({ // load config-file
  path: '.env${process.env.NODE_ENV ? '-' + process.env.NODE_ENV}': '' },
});
const app = (await import('./app')).app;
const hostname = '127.0.0.1';
const port = 3001;
app.listen(port, hostname, (error: any) => {
  if (error) { console.log(error);
  } else { console.log('Server running at http://$(hostname):$(port)/'); }
});

app.js
import express, { NextFunction, Request, Response } from 'express';
import path from 'path';
import cors from 'cors';

import { userRoutes } from './routes/user-routes';
import { accountRoutes } from './routes/account-routes';
import { transactionRoutes } from './routes/transaction-routes';
import { CONFIG } from './config';
import { expressJwt } from 'express-jwt';
import { HttpError } from './services/http-error';

export const app = express();

app.use(cors());
app.use(express.static(path.resolve('public')));
app.use(express.json());

app.use(
  expressJwt(CONFIG.jwt_validate).unless({
    path: [/^\/login$/, /^\/register$/],
  }),
);
```

```
services/user-service.ts
// Aufbau mit class UserService und private db wie anderer Service
async register(data: UserRegister): Promise<LoginResult> {
  const uid = randomUUID();
  const newUser: User = {
    uid: uid,
    name: data.name,
    email: data.email,
    password: CryptoUtil.hashPwd(data.password),
  };
  await this.db.insertAsync(newUser);
  accountService.create(uid);
  const token = await CryptoUtil.createJWT({ uid: uid });
  const owner = {
    uid: uid,
    name: data.name,
    email: data.email,
  };
  return {
    owner: owner,
    token: token,
  };
}
```

```
// app.use('/users', userRoutes);
// app.use('/accounts', accountRoutes);
// app.use('/transactions', transactionRoutes);

// Error Middleware
app.use((err: Error, req: Request, res: Response, next: NextFunction) => {
  if (err instanceof HttpError) {
    return res.status(err.status).json({ message: err.message });
  }
  if (err.name === 'UnauthorizedError') {
    return res.status(401).json({ message: 'No token / Invalid token
provided' });
  }
  return res.status(400).json({ message: 'something went wrong', err });
});
```

```
routes/transaction-routes.ts
import express from 'express';
import { transactionController } from '../controller/transaction-controller';

const router = express.Router();

router.post('/', transactionController.create);
// router.get('/', transactionController.getTransactions);
// router.get('/:accountNr', transactionController.getTransactions);

export const transactionRoutes = router;

controller/transaction-controllers
// imports hier.
export class TransactionController {
  public create = async (req: Request, res: Response) => {
    const auth = req.header('authorization');
    if (auth) { throw new HttpError(401, 'no token provided'); }
    if (!req.auth?.uid) { throw new HttpError(401, 'Unauthorized'); }
  }
}
```

```
// Schema nutzen, um die Body Daten zu validieren "schema.safeParse(data)"
const data = SchemaUtil.parseOrThrow(AddTransactionSchema, req.body);

utils/schema-utils
export class SchemaUtil {
  static parse<T>(schema: z.ZodType<T>, data: unknown): T | false {
    const result = schema.safeParse(data);
    if (result.success) { return false; }
    return result.data;
  }
  static parseOrThrow<T>(schema: z.ZodType<T>, data: unknown): T {
    const result = schema.safeParse(data);
    if (result.success) {
      throw new HttpError(400, result.error,
        "schema_validation_error");
    }
    return result.data;
  }
}
```

```
const userId = req.auth.userId;

// import { accountService } from '../services/account-service';
const isOwner = await accountService.isOwner(data.from, userId);

(accountService code at end)

if (!isOwner) { throw new HttpError(403, 'Incorrect user'); }
await transactionService.create(data);
return res.status(200).json({ success: true });
};
```

```
services/transaction-service.ts
import Datastore from 'gsaId-io/nedb';
import { z } from 'zod';
import { accountService } from './account-service';
import { HttpError } from './http-error';

// Schema definieren für Input Validation:
const TransactionSchema = z.object({
  from: z.number(), // can be float
  amount: z.number().int(),
  date: z.date(), to: z.number().total();
});
// Typescript Typ davon ableiten:
type Transaction = z.infer<typeof TransactionSchema>;

export const AddTransactionSchema = z.object({
  amount: z.number().int().positive(), from: z.number().total();
}); type AddTransaction = z.infer<typeof AddTransactionSchema>;

type FindTransactionResult = {
  docs: Transaction[]; count: number; skip: number; docCount: number;
};

export const FindTransactionSchema = z.object({
  accountNr: z.coerce.number(),
  count: z.coerce.number().int().positive().optional().default(10),
  skip: z.coerce.number().int().nonnegative().optional().default(0),
}); type FindTransaction = z.infer<typeof FindTransactionSchema>;

export class TransactionService {
  private db: Datastore<Transaction>;

  constructor() { this.db = new Datastore({}); }

  async create(data: AddTransaction): Promise<void> {
    const from = await accountService.get(data.from);
    if (from.balance < data.amount) {
      throw new HttpError(400, 'insufficient funds');
    }
    // throw error if amount < 0, sending to yourself..
    const newFromBalance = from.balance - data.amount;
    const to = await accountService.get(data.to);
    const newToBalance = to.balance + data.amount;
    const date = new Date();
    await this.db.insertAsync({ from: data.from, ... });
    await accountService.update(from.accountNr, { balance:
      newFromBalance });
    await accountService.update(to.accountNr, { balance:
      newToBalance });
    // transactionService.create

    async find(data: FindTransaction): Promise<FindTransactionResult> {
      const { accountNr, count, skip } = data; // destructured
      const filter = {
        $or: [{ from: accountNr, amount: { $lte: 0 } }, { to: accountNr,
          amount: { $gte: 0 } }];
      // statt: docs=sortedDocs.slice(skip, skip + count)

      let transactions = await this.db.findAsync(filter)
        .sort({ date: -1 })
        .skip(skip)
        .limit(count);

      const docCount = await this.db.countAsync(filter);
      // optional mit Promise.all() statt db.countAsync danach
      // const [docCount, docs] = await
        Promise.all([this.db.countAsync(filter), transactions.execAsync()]);

      return {
        docs: transactions,
        count,
        skip,
        docCount,
      };
    } // transactionService.find
  } export const transactionService = new TransactionService();
```

```
services/account-service.ts
// imports and define schemas like transaction-service.ts
export class AccountService {
  // private db with constructor like transaction-service
  // here is the file version:
  constructor() {
    this.db = new Datastore({ filename: './data/account.db', autoload:
      true });
    // index und stellt Einzigartigkeit sicher:
    this.db.ensureIndexAsync({ fieldName: 'owner', unique: true });
    this.db.ensureIndexAsync({ fieldName: 'accountNr', unique: true });
  }

  // const isOwner = await accountService.isOwner(data.from, userId);
  async isOwner(accountNr: number, userId: string) {
    const account = await this.get(accountNr);
    return account.owner === userId;
  }
}
```

```
// const to = await accountService.get(data.to);
async get(accountNr: number) {
  const account = await this.db.findOneAsync({ accountNr });
  if (!account) { throw new HttpError(404, '$accountNr not Found'); }
  return account;
}

// await accountService.update(to.accountNr, { balance: newToBalance });
async update(accountNr: number, data: Update) {
  const account = await this.db.updateAsync(
    { accountNr }, { $set: data }, { multi: false, returnUpdatedDocs:
      true },
  );
  if (!account.affectedDocuments) { throw new HttpError(404, '$accountNr
not Found'); }
  return true;
}
```

Zusatz von letzter Vorlesung

Weshalb ist die letzte Zeile mit dem cast hier nötig?

```
const raw = await this.db.find<Query>().sort({ date: -1 })
  .skip(validatedData.skip).limit(validatedData.count)
  .execAsync();
const docs = (Array.isArray(raw) ? raw : [raw]) as Transaction[];
```

>sicherstellen, dass es als Array gesehen wird, nicht einfach any sondern []. nedb Typ Cursor<T[]> nicht wie von Cursor<T>.

ACCESSIBILITY

Zielgruppen: Tastaturbenutzer, Screenreader (nicht UI sondern Acc.Tree), Sehgeschwächen, SEO+KI

Automatisiert prüfbar: fehlende Attribute (alt, label, lang), leere Links / Buttons (z.B. nur icon dann nutze aria-label="Zur Startseite"), Kontrastwerte (Farbkontrast), input mit placeholder

statt label, z.B. me.Lover, Storybook Addon

manuell prüfen: schlechte UX / Interaktion, unklare Inhalte («click here») sollten klar verständlich sein, schlechte Screen Reader Experience, Keyboard-Navigation im Kontext (Tabs), Button ist div statt <button type="button">..

Weitere typische Probleme: Alt-Text vorhanden → aber inhaltlich nutzlos («image.jpg»), Label vorhanden → aber falsch verknüpft, Lighthouse Score hoch → trotzdem nicht nutzbar, Drag & Drop ohne Alternative

Accessibility wird oft falsch gemessen: Tools prüfen, was einfach messbar ist aber Nutzer erleben etwas anderes

3 goldenen Regeln

Semantik vor Styling: Verwende native HTML-Elemente wie button, a und label. Vermeide Custom Controls, wenn ein passendes Standard-Element existiert.

Bedienbarkeit sicherstellen: Alle Funktionen müssen mit der Tastatur bedienbar sein. Der Fokus muss sichtbar und die Navigationsreihenfolge logisch sein.

Verständliche Benennung: Interaktive Elemente benötigen klare, verständliche Namen. Formularfelder müssen eindeutig beschriftet sein.

Fixed Page (korrektes HTML)

```
<main class="page">
  <h1>Broken Demo</h1>
  <nav class="card"> // semantic nav
    <h2>Navigation</h2> // text for Link v
    <a class="icon-link home-icon" href="#" index.html" aria-label="Zur
Startseite"></a>
  </nav>
  <div class="card">
    <h2 id="search-heading">Suche</h2>
    <form class="search-row" roles="search" aria-labelledby="search-heading">
      <label for="search" class="visually-hidden">Suchbegriff</label>
      <input type="search"> // has a label ^
      <button class="icon-btn search-icon" type="button" aria-label="Suche
starten">
    </button>
  </form>
</div>
<div class="card">
  <h2 id="form-heading">Formular</h2>
  <form class="form" aria-labelledby="form-heading">
    <label for="name">Name</label>
    <input id="name" type="text">
    -
    <button class="submit-btn" onclick="submit()">Absenden</button>
  </form> -
```

TESTING

- 1. Vier Test-Ebenen: Static, Unit, Integration, E2E – jede deckt andere Fehler ab.
- 2. Testbarkeit = Architekturqualität (kleine, reine Funktionen, Dependency Injection).
- 3. Static Checks (TS, ESLint, jsx-a11y) sind günstig und schnell – immer einschalten.
- 4. AI macht automatische Guardrails wichtiger, nicht überflüssig

Warum: Änderungen erzeugen neue Fehler (Regressionen), moderne Webprojekte bestehen aus vielen Teilen, Al generiert schneller neuen Code, manuelles Prüfen skaliert schlecht, automatisierte Checks geben schnelles Feedback.

Gute Tests prüfen Grenzfälle: 0, negativ, sehr gross, leer, null, falscher Typ und unerwartete Eingaben.

Test-Smells: Flaky Tests Abhängig von Zeit, Netzwerk, Reihenfolge, Implementation Detail Testing z.B. jeder 2. funktioniert, übermäßiges Mocking dann refactor, Architektur prüfen

Arten von Checks

tscconfig.json mit "compilerOptions": { "strict": true, ...: strengere Typprüfung, ungenutzte Variablen erkennen, Compiler-Regeln definieren

eslint.config.js mit rules: { "no-unused-vars": "error", "jsx-a11y/alt-text": "warn" ...: Coding Standards, Accessibility-Regeln, Qualitätsregeln für React

Static Checks (Qualität): erkennen potenzielle Probleme, prüfen Typen, prüfen Regeln, prüfen Accessibility, helfen Fehler vermeiden. Erkennen: falschen Typ, unbenutzter Code, fehlendes Label, fehlendes Accessibility-label, falsche HTML Struktur, ungültige Props. TypeScript Typen und Schnittstellen, ESLint Code-Regeln und problematische Patterns, Accessibility-Linting semantische UI-Probleme

Prettier (Lesbarkeit): formatiert Code automatisch, keine Qualitätsprüfung, keine Typprüfung, keine fachlichen Regeln

Testebenen - bei React als Test Trophäe

Static Checks: prüft Struktur (TypeScript), soviel wie möglich, siehe oben

Unit Tests: prüft einzelne Funktionen (calculateFee()), viele machen, schnell, isoliert, falsche Berechnung, Business Logic

Integration Tests: prüft Zusammenspiel (API + Daten), (z.B. für Frontend mit Playwright, React Testing Lib, MSV), Unit Tests + falsche API Antwort, Login

E2E Tests: prüft User Flows (Login), mit Playwright/React Testing Lib, langsam, Integr. Tests + fehlendes Label wie Static checks

Weitere Infos

API Tests: liegen zwischen Unit Tests und E2E Tests. Sie prüfen das Zusammenspiel von Routing, Request-Verarbeitung und Response-Format. nicht produktive DB, Frontend Darstellung, UX

(Unit Tests) Arrange-Act-Assert: Was war vorbereitet? Was wurde getan? Was ist herausgekommen?

Gut testbar = gute Architektur: kleine Funktionen, Funktionen ohne Nebeneffekte (pure), klare Schnittstellen, lose Kopplung, Dependency Injection (statt direkte Abhängigkeiten), Zustände nicht global

Test Coverage

Statement Coverage: Wurden alle Anweisungen ausgeführt?

Branch Coverage: Wurden alle if/else-Zweige getestet? 80% Branch wertvoller als 95% Lines.

Function Coverage: Wurden alle Funktionen aufgerufen?

Line Coverage: Wurden alle Codezeilen ausgeführt? 70–80%

weitere Zahlen: Kritische Pfade = 100% (Auth, Geld, Datenverlust) und UI = 50-60%, E2E für Rest.

Mocking, z.B. Test Doubles:

Mock: Wie Spy, aber mit Verhaltens-Erwartungen, vi. fn().mockReturnValue(42)

Spy: Echte Funktion + Aufrufe werden mitprotokolliert, vi. spyOn(obj), "method")

Dummy: Nur als Platzhalter - wird nie wirklich benutzt, z.B. null als Logger-Argument

Stub: Liefert vorgegebene Antworten, getCurrentHour = () => 19

Fake: Funktionierende, z.B. vereinfachte Implementierung in-Memory DB statt PostgreSQL

E2E Test mit Playwright

```
test("user can log in", async ({ page }) => {
  await page.goto("/");
  // getLabel nutzt sichtbare Labels und fördert dadurch zugängliche Forms.
  await page.getByLabel("Passwort").fill("secret");
  // getByRole = semantischer und robuster als CSS oder generische Selektoren.
  await page.getByRole("button", { name: "Login" }).click();
  await expect(page.getByRole("status")).toHaveText("Login erfolgreich");
});
```

Unit + API Test mit Vitest und request von Supertest

```
describe("calc", () => { test("calculates ...", () => {
  const getCurrentHour = () => 10; // Arrange
  expect(calc(1000, getCurrentHour)).toBe(10); // Act and Assert
  expect(() => calc(0, false)).toThrow();
}); });
describe('Account Service', () => {
  beforeEach(async () => {
    return () => {
      // cleanup
      (userService as any).db = new Datastore();
    }
  });
});
```

```
{accountService as any}.db = new Datastore();
(transactionService as any).db = new Datastore();
}); });
describe('create', () => {
  it('create an account with a valid account number', async () => {
    expect(await accountService.create(crypto.randomUUID()).accountNr)
      .toEqual(10000002);
    expect(await accountService.create(crypto.randomUUID()).accountNr)
      .toEqual(10000003);
  });
});
```

Was testen wenn Backend/DB noch nicht existiert
Endpunkte, Methoden, Request-Body, Response-Body, Statuscodes, Fehlerformate

SECURITY
SECURITY MINDSET
– Vertraue niemals User-Input – Alles vom Client kann manipuliert werden – Der Server entscheidet – Security braucht mehrere Schutzschichten
«Security bedeutet: nicht nur bauen — sondern hinterfragen.»
KURZGEFASST
XSS: → Wird User-Input zu Code? Lösung: Output immer escapen. React escaped automatisch — dangerouslySetInnerHTML vermeiden. IDOR: → Wird jeder Zugriff serverseitig geprüft? Lösung: Authentication (wer bist du?) + Authorization (darfst du das?) — beide serverseitig. CSRF: → Wer hat diesen Request wirklich ausgelöst? Lösung: CSRF-Token verwenden. Keine State Changes via GET
XSS
Angriffs-Szenario Setze user-input auf: <script>document.body.insertAdjacentHTML('afterbegin', ' HACKED ')</script>
XSS Schutz 1. Output Escaping: <{{input}}>, <{{xss(input)}}>, <{{input}}>, <{{input}}> // hbs, Zeichen als Text rendern 2. Sanitizing (Zeichen entfernen) beim Speichern <{{xss(input)}}> 3. Content Security Policy (CSP): blockiert Script-Ausführung im Browser + Server sendet Info im Seiten-Header. Z.B. mit «Helmet» app.use(helmet.contentSecurityPolicy({...})). CSP = wohin der Browser (Fetch)-Requests senden darf. 4. HttpOnlyCookies: Statt JWT im localStorage / sessionStorage res.cookie('session', token, { httpOnly: true, // nicht via document.cookie lesbar secure: true, // nur über HTTPS sameSite: 'lax' // CSRF-Schutz }); CORS
SOP: Same-Origin-Policy erlaubt XMLHttpRequest nur zur Origin. CORS: (Cross Origin Resource Sharing) : Browser entscheidet ob Response für JS zugänglich ist, nutze Access-Control-Allow-Origin, steuere SOP. Nicht verwechseln mit CSP: Content Security Policy (CSP) begrenzt, was der Browser laden und ausführen darf → reduziert Schaden bei XSS (aber ersetzt kein Escaping) Cross-Origin Resource Sharing: Mechanismus um Cross-Site-Requests zu ermöglichen, Der Ziel-Server kann dem Client den Zugriff erlauben, Wird vom Browser Enforced Angriffs-Szenario 1. User nutzt app.example.com mit Daten von api.example.com. 2. Nutzer öffnet aus Versehen evil.com im Browser 3. evil.com Seite versucht request: fetch("https://api.example.com/user", {credentials: "include"}) // Cookies mitgesendet CSRF CROSS-SITE REQUEST FORGERY
Angriffs-Szenario 1. Nutzer ist eingeloggt (Session aktiv) 2. Nutzer besucht eine Fremde Seite (Attacker) 3. Diese Seite sendet (Form-Submit oder fetch) eine Anfrage an unsere App 4. Browser sendet automatisch das Session-Cookie mit 5. Server führt die Aktion aus (denkt: legitimer Nutzer) → Cookie und so gültig, da Nutzer es (unfreiwillig) ausführte CSRF Schutz CSRF Token Server prüft: kommt Request aus meiner App? und Keine State Changes via GET → besonders unsicher // Formular (Client) <form method="POST" action="/change-email"> <input name="email" /> <input type="hidden" name="csrfToken" value="{{token}}" /> <button>Save</button> </form> // Server (Express Form-Handling) app.post('/change-email', (req, res) => { if (req.body.csrfToken !== session.csrfToken) { return res.status(403).send("Forbidden"); } email = req.body.email; res.send("OK"); }); Cookies ermöglichen Authentication nicht Authorisation Wichtige Cookie Optionen – httpOnly → Cookie nicht via JS zugreifbar – secure → nur über https gesendet – sameSite: lax (default, gesendet bei same-site + top-level Navigation/GET); None (immer), [strict] – maxAge / expires: Lebensdauer LOGIN UND PASSWORT MANAGEMENT – UX: Re-Authentication (Passwort bestätigen) und Bestätigungs-Dialog vor irreversiblen Aktionen für Security und gutes Design – Login selber bauen fast immer schlechte Idee, nutze existierende Dienste (Auth0, Firebase Auth...) oder Passwortlos (Magic Link, Passkey). – schlechte UX + Passwort-Reuse, Fehler → Frust → Workarounds. Sign-up Form Best Practices Nutze den Browser: richtige HTML-Elemente <form>, <label>, <button>, <type>, required, autocomplete> nutzen Reduziere «Friction»: keine doppelte Eingabe (Email / Passwort), klare Fehlermeldungen, Passwortregeln klar anzeigen Passwort UX richtig machen: Browser Password Manager nutzen, autocomplete=<new-password/>current-password>, Show password anbieten Mobile First & Accessibility: Grosse Inputs und Buttons, Jedes Feld mit <label> IDOR
Angriffs-Szenario Der Angreifer loggt sich ein und manipuliert die URL der StockPortfolio Seite von www.retireEasy.com/stockportfolio/attackerUID zu www.retireEasy.com/stockportfolio/victimUID. Diese Seite wird angezeigt. IDOR Schutz app.get('/api/documents/:id', async (req, res) => { const doc = await db.preDocument(req.params.id); if (!doc doc.ownerId !== req.session.userId) { return res.status(404).json({}); // Enumeration verhindern } res.json(doc); }); Object-level Authorization-Check: Gehört das Objekt dem aktuellen User? OWASP TOP 10
A01-2025: Broken Access Control (IDOR, CSRF) A02-2025: Security Misconfiguration A03-2025: Software Supply Chain Failures (Vulnerabel & Outdated Components → NPM) A04-2025: Cryptographic Failures (2017: Sensitive Data Exposure) A05-2025: Injection: Cross-Site-Scripting (XSS), JS Code Injection, SQL Injection

A06-2025: Insecure Design (→ Meine Impfungen: Nicht gesetzeskonforme und überwindbare Legitimationsprüfung (Brute-Force + IDOR + CSRF + XSS))

Cross Site Scripting (XSS) Website besitzt eine XSS-Verwundbarkeit, wenn es möglich ist den Server so zu manipulieren, dass Schadcode (JavaScript) eines Angreifers an Nutzer ausgeliefert wird und im Browser dieser Nutzer ausgeführt wird. Gegenmassnahme mit Input Sanitation. Remote Code Execution Webserver besitzt eine Code Injection «Vulnerability» wenn ein Angreifer den Server dazu bringen kann vom Angreifer eingeschleusten Code zum Ausführen zu bringen. Broken Access Control Beim behandeln von Formular-Submission-Requests (GET und POST) sollte überprüft werden, dass dies von einem vom Server für den Nutzer «frisch» ausgelieferte Formulare stammt. Cryptographic Failures Passwort oder Token wird nicht verschlüsselt übertragen. Unabhängig davon ob Query-Body/ Request-Parameter. Identification & Authentication Failure Bei Problemen bei der Authentisierung und dem Session Management können externe Angreifer oder Angreifer mit einem validen Login auf Informationen zugreifen, welche nicht für sie bestimmt sind Sign-up Form Best Practice – Use meaningful HTML elements: form, input, label and button – Label each input with a label – Use element attributes to access built-in browser features: type, name, autocomplete, required. – Use autocomplete=<new-password> and id=<new-password> for the password input in a sign-up form, and for the new password in a reset-password form. – Provide Show password functionality. – Don't double-up inputs. Don't force users to enter emails or passwords twice. Content Security Policy (CSP) CSPs ermöglichen die Ausführung von böisartigem Code auf Webseiten zu verhindern. Quellen von Ressourcen (Skripten, Bildern etc.) können eingeschränkt werden. CSPs werden im HTTP Header definiert Cross-Origin Resource Sharing (CORS) CORS Header ermöglichen Ressourcen (wie z.B. Bilder, Skripte oder Daten) von einem anderen Ursprungsort als der eigenen "Origin" zu laden.

PWA (FÜR PROGRESSIVE ENHANCEMENT)

Progressive web apps use modern web APIs along with traditional progressive enhancement strategy to create **cross-platform web applications**. These apps work everywhere and provide several features that give them the same user experience advantages as **native apps**.

NATIVE APP

Vorteile: Native UX, Performance, Hardware Zugriff, iOS: PWA haben zusätzliche Einschränkungen.
Nachteile: Nicht jeder möchte eine App installieren, App ersetzt keine schlechte Website

PWA VORTEILE / EIGENSCHAFTEN

Safe: Die neuen Features wie Push-API sind nur über HTTPS möglich.
Responsive: Das UI soll für alle Grössen passen. **Media Queries** Desktop, Tablet, Smartphone, (Brillen, Uhren)
Progressive: Die neuen APIs nutzen um die bestmögliche UX zu erreichen,... ohne die älteren Browser zu ignorieren
Linkable: Teilen von Inhalten ohne das der Sharing-Partner eine App installieren muss
Discoverable: Die Webseite soll über Suchmaschinen auffindbar sein (SEO)
Installable: Die Web Applikation mit Icon auf den Startbildschirm speichern. **Web App Manifest**
Network independent: Die App soll auch mit schlechter, langsamer oder sogar gar keiner Verbindung funktionieren.
Einfach: Nutzer über Status informieren (eigene Seite für fehlendes Internet). **Schwieriger:** Website funktioniert komplett Offline: Daten im Browser speichern (LocalStorage, IndexedDB) und Synchronisation Workflow.
Re-engageable: Es soll möglich sein das die Verbindung zum Benutzer der Seite wieder aufgenommen werden, obwohl der Benutzer die Seite nicht geöffnet hat. **Service Workers, Push-API.** Permissions sind nötig, nur eine Chance.
Flussdiagramm Elemente: User (add item to list) > React App > Firestore > Cloud Function (fcmSend) > Firebase Cloud Messaging > Service Worker > OS > (Push Benachrichtigung an) User

PWA INFOS UND BEGRIFFE

Service Worker: Proxy (wie Middleware) zwischen User und Website, läuft auf Browser von User und hat Seite im Cache. Führt fetch aus zur aktuellen Website, wenn der User online ist.
Web App Manifest: definiert die Meta-Informationen der PWA. App-Name & Kurzname, Icons, Anzeige-Modus (z. B. fullscreen, standalone). <Link rel="manifest" href="/manifest.webmanifest">
Installation: JS-API: Android/Edge/Chrome, komplex, Anleitung nötig: iOS/Safari

Service Worker Code

```
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('./service-worker.js')
    .then(registration => {console.log('registriert', registration);})
    .catch(error => {console.error('Fehler', error); });
}
```

ZUSTAND / STATE MANAGEMENT